

STAR Grafana Dashboard JSON

Source Listing

Auto-generated syntax-highlighted listing

Version 1.1

April 2026

MIT License

STAR Development Team
Locked Inc.

Generated by `scripts/codedump-pdf.sh`.

Contents

1	(root)	2
	grafana-star-dashboard.json	2

1 (root)

grafana-star-dashboard.json

```
1 {
2   "annotations": { "list": [] },
3   "editable": true,
4   "fiscalYearStartMonth": 0,
5   "graphTooltip": 1,
6   "id": null,
7   "links": [],
8   "panels": [
9     {
10      "type": "row",
11      "title": "SLAM map",
12      "gridPos": { "h": 1, "w": 24, "x": 0, "y": 0 },
13      "id": 700,
14      "collapsed": false,
15      "panels": []
16    },
17    {
18      "type": "text",
19      "title": "/map -- live top-down view (served by star_simple_bridge @ :9101)",
20      "id": 29,
21      "gridPos": { "h": 14, "w": 24, "x": 0, "y": 1 },
22      "options": {
23        "mode": "html",
24        "content": "<div
↵ style=\"display:flex;justify-content:center;align-items:center;height:100%;background:#111;\"><img
↵ src=\"http://100.64.0.6:9101/map.png?t=${_from}\"
↵ style=\"max-width:100%;max-height:100%;image-rendering:pixelated;\" alt=\"no /map yet -- is slam_toolbox
↵ active?\"></div>"
25      }
26    },
27    {
28      "type": "row",
29      "title": "Motors",
30      "gridPos": { "h": 1, "w": 24, "x": 0, "y": 15 },
31      "id": 100,
32      "collapsed": false,
33      "panels": []
34    },
35    {
36      "type": "gauge",
37      "title": "FL velocity",
38      "id": 1,
39      "gridPos": { "h": 7, "w": 6, "x": 0, "y": 1 },
40      "targets": [{ "expr": "star_motor_velocity_mps{wheel=\"fl\"}", "refId": "A", "legendFormat": "" }],
41      "fieldConfig": {
42        "defaults": {
43          "unit": "velocityms",
44          "min": -0.3,
45          "max": 0.3,
46          "thresholds": {
47            "mode": "absolute",
48            "steps": [
49              { "color": "blue", "value": null },
50              { "color": "green", "value": -0.25 },
51              { "color": "yellow", "value": 0.2 },
52              { "color": "red", "value": 0.25 }
53            ]
54          }
55        }
56      },
57      "options": { "orientation": "auto", "showThresholdLabels": false, "showThresholdMarkers": true }
58    },
59    {
60      "type": "gauge",
61      "title": "FR velocity",
62      "id": 2,
63      "gridPos": { "h": 7, "w": 6, "x": 6, "y": 1 },
64      "targets": [{ "expr": "star_motor_velocity_mps{wheel=\"fr\"}", "refId": "A", "legendFormat": "" }],
65      "fieldConfig": {
66        "defaults": {
67          "unit": "velocityms",
68          "min": -0.3,
69          "max": 0.3,
70          "thresholds": {
71            "mode": "absolute",
```

```

72         "steps": [
73             { "color": "blue", "value": null },
74             { "color": "green", "value": -0.25 },
75             { "color": "yellow", "value": 0.2 },
76             { "color": "red", "value": 0.25 }
77         ]
78     }
79 }
80 }
81 },
82 {
83     "type": "gauge",
84     "title": "BL velocity",
85     "id": 3,
86     "gridPos": { "h": 7, "w": 6, "x": 12, "y": 1 },
87     "targets": [{ "expr": "star_motor_velocity_mps{wheel=\"bl\"}", "refId": "A", "legendFormat": "" }],
88     "fieldConfig": {
89         "defaults": {
90             "unit": "velocityms",
91             "min": -0.3,
92             "max": 0.3,
93             "thresholds": {
94                 "mode": "absolute",
95                 "steps": [
96                     { "color": "blue", "value": null },
97                     { "color": "green", "value": -0.25 },
98                     { "color": "yellow", "value": 0.2 },
99                     { "color": "red", "value": 0.25 }
100                 ]
101             }
102         }
103     }
104 },
105 {
106     "type": "gauge",
107     "title": "BR velocity",
108     "id": 4,
109     "gridPos": { "h": 7, "w": 6, "x": 18, "y": 1 },
110     "targets": [{ "expr": "star_motor_velocity_mps{wheel=\"br\"}", "refId": "A", "legendFormat": "" }],
111     "fieldConfig": {
112         "defaults": {
113             "unit": "velocityms",
114             "min": -0.3,
115             "max": 0.3,
116             "thresholds": {
117                 "mode": "absolute",
118                 "steps": [
119                     { "color": "blue", "value": null },
120                     { "color": "green", "value": -0.25 },
121                     { "color": "yellow", "value": 0.2 },
122                     { "color": "red", "value": 0.25 }
123                 ]
124             }
125         }
126     }
127 },
128 {
129     "type": "gauge",
130     "title": "FL duty",
131     "id": 5,
132     "gridPos": { "h": 7, "w": 6, "x": 0, "y": 8 },
133     "targets": [{ "expr": "star_motor_duty_percent{wheel=\"fl\"}", "refId": "A" }],
134     "fieldConfig": {
135         "defaults": {
136             "unit": "percent",
137             "min": -100,
138             "max": 100,
139             "thresholds": {
140                 "mode": "absolute",
141                 "steps": [
142                     { "color": "green", "value": null },
143                     { "color": "yellow", "value": 60 },
144                     { "color": "red", "value": 85 }
145                 ]
146             }
147         }
148     }
149 },
150 {

```

```

151     "type": "gauge",
152     "title": "FR duty",
153     "id": 6,
154     "gridPos": { "h": 7, "w": 6, "x": 6, "y": 8 },
155     "targets": [{ "expr": "star_motor_duty_percent{wheel=\"fr\"}", "refId": "A" }],
156     "fieldConfig": {
157       "defaults": {
158         "unit": "percent", "min": -100, "max": 100,
159         "thresholds": { "mode": "absolute",
160           "steps": [
161             { "color": "green", "value": null },
162             { "color": "yellow", "value": 60 },
163             { "color": "red", "value": 85 }
164           ]
165         }
166       }
167     },
168   },
169   {
170     "type": "gauge",
171     "title": "BL duty",
172     "id": 7,
173     "gridPos": { "h": 7, "w": 6, "x": 12, "y": 8 },
174     "targets": [{ "expr": "star_motor_duty_percent{wheel=\"bl\"}", "refId": "A" }],
175     "fieldConfig": {
176       "defaults": {
177         "unit": "percent", "min": -100, "max": 100,
178         "thresholds": { "mode": "absolute",
179           "steps": [
180             { "color": "green", "value": null },
181             { "color": "yellow", "value": 60 },
182             { "color": "red", "value": 85 }
183           ]
184         }
185       }
186     },
187   },
188   {
189     "type": "gauge",
190     "title": "BR duty",
191     "id": 8,
192     "gridPos": { "h": 7, "w": 6, "x": 18, "y": 8 },
193     "targets": [{ "expr": "star_motor_duty_percent{wheel=\"br\"}", "refId": "A" }],
194     "fieldConfig": {
195       "defaults": {
196         "unit": "percent", "min": -100, "max": 100,
197         "thresholds": { "mode": "absolute",
198           "steps": [
199             { "color": "green", "value": null },
200             { "color": "yellow", "value": 60 },
201             { "color": "red", "value": 85 }
202           ]
203         }
204       }
205     },
206   },
207   {
208     "type": "timeseries",
209     "title": "Wheel velocities over time",
210     "id": 9,
211     "gridPos": { "h": 8, "w": 12, "x": 0, "y": 15 },
212     "targets": [{ "expr": "star_motor_velocity_mps", "refId": "A", "legendFormat": "{{wheel}}" }],
213     "fieldConfig": { "defaults": { "unit": "velocityms" } },
214     "options": { "legend": { "displayMode": "list", "placement": "bottom", "showLegend": true } }
215   },
216   {
217     "type": "timeseries",
218     "title": "Wheel duty over time",
219     "id": 10,
220     "gridPos": { "h": 8, "w": 12, "x": 12, "y": 15 },
221     "targets": [{ "expr": "star_motor_duty_percent", "refId": "A", "legendFormat": "{{wheel}}" }],
222     "fieldConfig": { "defaults": { "unit": "percent" } }
223   },
224   {
225     "type": "stat",
226     "title": "Motor faults (any != 0 is bad)",
227     "id": 11,
228     "gridPos": { "h": 4, "w": 24, "x": 0, "y": 23 },
229     "targets": [{ "expr": "star_motor_fault_byte", "refId": "A", "legendFormat": "{{wheel}}" }],

```

```

230     "fieldConfig": {
231       "defaults": {
232         "thresholds": {
233           "mode": "absolute",
234           "steps": [
235             { "color": "green", "value": null },
236             { "color": "red", "value": 1 }
237           ]
238         }
239       }
240     },
241     "options": { "colorMode": "background", "graphMode": "none", "textMode": "value_and_name" }
242   },
243
244   {
245     "type": "row",
246     "title": "IMU",
247     "gridPos": { "h": 1, "w": 24, "x": 0, "y": 27 },
248     "id": 200,
249     "collapsed": false
250   },
251   {
252     "type": "gauge",
253     "title": "Roll",
254     "id": 12,
255     "gridPos": { "h": 7, "w": 8, "x": 0, "y": 28 },
256     "targets": [{ "expr": "star_imu_orientation_deg{axis=\"roll\"}", "refId": "A" }],
257     "fieldConfig": { "defaults": { "unit": "degree", "min": -180, "max": 180 } }
258   },
259   {
260     "type": "gauge",
261     "title": "Pitch",
262     "id": 13,
263     "gridPos": { "h": 7, "w": 8, "x": 8, "y": 28 },
264     "targets": [{ "expr": "star_imu_orientation_deg{axis=\"pitch\"}", "refId": "A" }],
265     "fieldConfig": { "defaults": { "unit": "degree", "min": -180, "max": 180 } }
266   },
267   {
268     "type": "gauge",
269     "title": "Heading",
270     "id": 14,
271     "gridPos": { "h": 7, "w": 8, "x": 16, "y": 28 },
272     "targets": [{ "expr": "star_imu_orientation_deg{axis=\"heading\"}", "refId": "A" }],
273     "fieldConfig": { "defaults": { "unit": "degree", "min": 0, "max": 360 } }
274   },
275   {
276     "type": "timeseries",
277     "title": "Gyro (deg/s)",
278     "id": 15,
279     "gridPos": { "h": 7, "w": 12, "x": 0, "y": 35 },
280     "targets": [{ "expr": "star_imu_gyro_dps", "refId": "A", "legendFormat": "{{axis}}" }],
281     "fieldConfig": { "defaults": { "unit": "rotdegs" } }
282   },
283   {
284     "type": "timeseries",
285     "title": "Linear acceleration (m/s^2)",
286     "id": 16,
287     "gridPos": { "h": 7, "w": 12, "x": 12, "y": 35 },
288     "targets": [{ "expr": "star_imu_accel_mps2", "refId": "A", "legendFormat": "{{axis}}" }],
289     "fieldConfig": { "defaults": { "unit": "accMS2" } }
290   },
291
292   {
293     "type": "row",
294     "title": "Odometry",
295     "gridPos": { "h": 1, "w": 24, "x": 0, "y": 42 },
296     "id": 300,
297     "collapsed": false
298   },
299   {
300     "type": "gauge",
301     "title": "Linear velocity",
302     "id": 17,
303     "gridPos": { "h": 6, "w": 8, "x": 0, "y": 43 },
304     "targets": [{ "expr": "star_odom_linear_mps", "refId": "A" }],
305     "fieldConfig": { "defaults": { "unit": "velocityms", "min": -0.3, "max": 0.3 } }
306   },
307   {
308     "type": "gauge",

```

```

309     "title": "Angular velocity",
310     "id": 18,
311     "gridPos": { "h": 6, "w": 8, "x": 8, "y": 43 },
312     "targets": [{ "expr": "star_odom_angular_radps", "refId": "A" }],
313     "fieldConfig": { "defaults": { "unit": "rads", "min": -2, "max": 2 } }
314 },
315 {
316     "type": "stat",
317     "title": "Odom pose (x, y, theta)",
318     "id": 19,
319     "gridPos": { "h": 6, "w": 8, "x": 16, "y": 43 },
320     "targets": [
321         { "expr": "star_odom_pose_x_m", "refId": "A", "legendFormat": "x (m)" },
322         { "expr": "star_odom_pose_y_m", "refId": "B", "legendFormat": "y (m)" },
323         { "expr": "star_odom_pose_theta_rad", "refId": "C", "legendFormat": "theta (rad)" }
324     ],
325     "options": { "colorMode": "value", "graphMode": "area", "textMode": "value_and_name" }
326 },
327
328 {
329     "type": "row",
330     "title": "LiDAR",
331     "gridPos": { "h": 1, "w": 24, "x": 0, "y": 49 },
332     "id": 500,
333     "collapsed": false
334 },
335 {
336     "type": "gauge",
337     "title": "Closest obstacle",
338     "id": 24,
339     "gridPos": { "h": 6, "w": 6, "x": 0, "y": 50 },
340     "targets": [{ "expr": "star_scan_closest_range_m", "refId": "A" }],
341     "fieldConfig": {
342         "defaults": {
343             "unit": "lengthm",
344             "min": 0,
345             "max": 3,
346             "thresholds": {
347                 "mode": "absolute",
348                 "steps": [
349                     { "color": "red", "value": null },
350                     { "color": "yellow", "value": 0.5 },
351                     { "color": "green", "value": 1.0 }
352                 ]
353             }
354         }
355     },
356 },
357 {
358     "type": "gauge",
359     "title": "Closest bearing (0deg=forward)",
360     "id": 25,
361     "gridPos": { "h": 6, "w": 6, "x": 6, "y": 50 },
362     "targets": [{ "expr": "star_scan_closest_bearing_deg", "refId": "A" }],
363     "fieldConfig": { "defaults": { "unit": "degree", "min": -180, "max": 180 } }
364 },
365 {
366     "type": "stat",
367     "title": "Obstacles < 1 m",
368     "id": 26,
369     "gridPos": { "h": 6, "w": 6, "x": 12, "y": 50 },
370     "targets": [{ "expr": "star_scan_obstacles_under_1m", "refId": "A" }],
371     "fieldConfig": {
372         "defaults": {
373             "thresholds": {
374                 "mode": "absolute",
375                 "steps": [
376                     { "color": "green", "value": null },
377                     { "color": "yellow", "value": 1 },
378                     { "color": "red", "value": 10 }
379                 ]
380             }
381         }
382     },
383     "options": { "colorMode": "background", "graphMode": "area" }
384 },
385 {
386     "type": "stat",
387     "title": "Scan range min / mean / max",

```

```

388     "id": 27,
389     "gridPos": { "h": 6, "w": 6, "x": 18, "y": 50 },
390     "targets": [{ "expr": "star_scan_range_m", "refId": "A", "legendFormat": "{{stat}}" }],
391     "fieldConfig": { "defaults": { "unit": "lengthm" } },
392     "options": { "textMode": "value_and_name", "colorMode": "value" }
393 },
394 {
395     "type": "row",
396     "title": "Bridge health",
397     "gridPos": { "h": 1, "w": 24, "x": 0, "y": 64 },
398     "id": 400,
399     "collapsed": true,
400     "panels": [
401         {
402             "type": "timeseries",
403             "title": "Line rate by kind (msg/s)",
404             "id": 20,
405             "gridPos": { "h": 8, "w": 12, "x": 0, "y": 50 },
406             "targets": [{ "expr": "rate(star_bridge_rx_lines_total[1m])", "refId": "A", "legendFormat":
↵ "{{$kind}}" }],
407         },
408         {
409             "type": "timeseries",
410             "title": "Parse errors (/s)",
411             "id": 21,
412             "gridPos": { "h": 8, "w": 12, "x": 12, "y": 50 },
413             "targets": [{ "expr": "rate(star_bridge_parse_errors_total[1m])", "refId": "A", "legendFormat":
↵ "{{$kind}}" }],
414             "fieldConfig": { "defaults": { "color": { "mode": "fixed", "fixedColor": "red" } } }
415         },
416         {
417             "type": "stat",
418             "title": "Serial reopens (cumulative)",
419             "id": 22,
420             "gridPos": { "h": 4, "w": 12, "x": 0, "y": 58 },
421             "targets": [{ "expr": "star_bridge_serial_reopens_total", "refId": "A" }],
422         },
423         {
424             "type": "stat",
425             "title": "RX bytes total",
426             "id": 23,
427             "gridPos": { "h": 4, "w": 12, "x": 12, "y": 58 },
428             "targets": [{ "expr": "star_bridge_rx_bytes_total", "refId": "A" }],
429             "fieldConfig": { "defaults": { "unit": "bytes" } }
430         }
431     ]
432 }
433 ],
434 "refresh": "1s",
435 "schemaVersion": 39,
436 "tags": ["star", "robot"],
437 "templating": { "list": [] },
438 "time": { "from": "now-2m", "to": "now" },
439 "timepicker": {},
440 "timezone": "",
441 "title": "STAR robot telemetry",
442 "uid": "star-robot-telemetry",
443 "version": 1,
444 "weekStart": ""
445 }

```

STAR Pi5 Cockpit + Systemd Units

Source Listing

Auto-generated syntax-highlighted listing

Version 1.1

April 2026

MIT License

STAR Development Team
Locked Inc.

Generated by `scripts/codedump-pdf.sh`.

Contents

1	(root)	2
	README.md	2
2	pi5	3
	pi5/etc/systemd/system/caddy.service	3
	pi5/etc/systemd/system/star-cockpit-api.service	3
	pi5/etc/systemd/system/star-slam-mvp.service	3
	pi5/opt/star_cockpit_api/server.py	4

1 (root)

README.md

```
1 # STAR infra -- Pi5-side system files
2
3 Non-ROS infra I manage on the Pi5 itself (outside the ROS workspace). These
4 are mirrored here so they live in version control, but the authoritative
5 copies are the ones deployed at the paths below.
6
7 ## Layout
8
9 ...
10 infra/pi5/
11 +-- opt/
12 |   +-- star_cockpit_api/
13 |   +-- server.py           -> /opt/star_cockpit_api/server.py
14 +-- etc/
15     +-- caddy/
16     |   +-- Caddyfile       -> /etc/caddy/Caddyfile
17     +-- systemd/system/
18         +-- caddy.service   -> /etc/systemd/system/caddy.service
19         +-- star-cockpit-api.service -> /etc/systemd/system/star-cockpit-api.service
20         +-- star-slam-mvp.service -> /etc/systemd/system/star-slam-mvp.service
21     ...
22
23 ## What each piece does
24
25 ### `star_cockpit_api` -- HTTP API backing the Grafana cockpit controls
26
27 A standalone Python rclpy node + HTTP server on `100.64.0.6:9102`.
28
29 Endpoints:
30
31 | Method | Path | Purpose |
32 |-----|-----|-----|
33 | GET | `/api/state` | `{autonomy, estop, speed, slam}` -- all cached |
34 | POST | `/api/autonomy/toggle` | flip `/star/autonomy_enable` |
35 | POST | `/api/estop/toggle` | flip `/star/estop` |
36 | POST | `/api/speed` | body `{"value":0.75}` -> publish `/star/speed_scale` |
37 | POST | `/api/cmd_vel` | body `{"linear":0.5,"angular":0}` -> publish `/cmd_vel` |
38 | POST | `/api/slam/start` | configure+activate `/slam_toolbox` |
39 | POST | `/api/slam/stop` | deactivate `/slam_toolbox` |
40
41 Subscribes to the latched `/star/state/*` topics so the toggle logic always
42 sees the supervisor's current view. A background thread refreshes the slam
43 lifecycle state every 4s (via `ros2 service call`) so `/api/state` is 0(1).
44
45 ### Systemd units
46
47 - **`caddy.service`** -- Caddy (on Pi5) listens on `100.64.0.6:443`, terminates
48   TLS with its own local CA (`tls internal`), fronts `star_simple_bridge` on
49   `:9101` and `star_cockpit_api` on `:9102`. `After=tailscaled.service` so it
50   waits for the tailscale IP to be assigned. `Restart=always`.
51
52 - **`star-cockpit-api.service`** -- runs `server.py` as the `star` user, with
53   ROS workspace sourced and Cyclone DDS config exported. Restart on failure.
54
55 - **`star-slam-mvp.service`** -- runs `scripts/slam-mvp.sh start` at boot
56   (after tailscale is up + an 8s USB-enumeration delay). Initial state is
57   MANUAL mode, e-stop cleared -- so the robot is safe on boot, operator
58   flips to AUTONOMY via the Grafana cockpit toggle.
59
60 ### `Caddyfile`
61
62 Pi5-side Caddy config. Uses `tls internal` -- no ACME, no Cloudflare, no
63 external CA. Laptop/phone clients install Caddy's local root CA once (see
64 `docs/ARCHITECTURE.md` -> "Client setup to use `star.local`"). No secrets.
65
66 ## Installing on a fresh Pi5
67
68 ```bash
69 # 1. Copy files (from repo root)
70 sudo cp infra/pi5/opt/star_cockpit_api/server.py /opt/star_cockpit_api/
71 sudo chmod +x /opt/star_cockpit_api/server.py
72
73 sudo cp infra/pi5/etc/caddy/Caddyfile /etc/caddy/Caddyfile
74 sudo cp infra/pi5/etc/systemd/system/*.service /etc/systemd/system/
75
```

```

76 # 2. Install Caddy binary with no plugins needed (tls internal only)
77 sudo curl -fsSL -o /usr/local/bin/caddy \
78 "https://caddyserver.com/api/download?os=linux&arch=arm64"
79 sudo chmod +x /usr/local/bin/caddy
80 sudo useradd --system --shell /usr/sbin/nologin --home-dir /var/lib/caddy caddy 2>/dev/null || true
81 sudo mkdir -p /var/log/caddy /var/lib/caddy
82 sudo chown caddy:caddy /var/log/caddy /var/lib/caddy
83
84 # 3. Enable + start
85 sudo systemctl daemon-reload
86 sudo systemctl enable --now caddy star-cockpit-api star-slam-mvp
87 ...
88
89 ## What is deliberately NOT mirrored here
90
91 - **Cluster-side k8s manifests** (traefik, robot-gateway, lichtblick, grafana
92   dashboard JSON). Those live in k3s's etcd; Grafana dashboard is pushed via
93   API. They can be exported separately if needed.
94 - **Any hostname or credentials** -- these system files use only `star.local`
95   and `100.64.0.6` (tailscale IP), nothing that identifies the operator.
96 - **The Caddy-gateway Caddyfile block on pve** that defines
97   `` -- that contains the public hostname and a bcrypt
98   password hash, and is managed on the homelab host, not on the Pi5.

```

2 pi5

pi5/etc/systemd/system/caddy.service

```

1 [Unit]
2 Description=Caddy
3 After=network-online.target tailscaled.service
4 Wants=network-online.target tailscaled.service
5
6 [Service]
7 User=caddy
8 Group=caddy
9 ExecStart=/usr/local/bin/caddy run --config /etc/caddy/Caddyfile
10 ExecReload=/usr/local/bin/caddy reload --config /etc/caddy/Caddyfile --force
11 Restart=always
12 RestartSec=5s
13 TimeoutStopSec=5s
14 LimitNOFILE=1048576
15 AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
16
17 [Install]
18 WantedBy=multi-user.target

```

pi5/etc/systemd/system/star-cockpit-api.service

```

1 [Unit]
2 Description=STAR cockpit HTTP API (Grafana toggle/speed backend)
3 After=network-online.target
4
5 [Service]
6 User=star
7 Group=star
8 ExecStart=/bin/bash -c "source /opt/ros/jazzy/setup.bash && source
9 ↪ /workspaces/STAR/star-ros2/install/setup.bash && export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp && export
10 ↪ CYCLONEDDS_URI=file:///workspaces/STAR/config/cyclonedds.xml && exec python3
11 ↪ /opt/star-cockpit-api/server.py"
12 Restart=always
13 RestartSec=3
14
15 [Install]
16 WantedBy=multi-user.target

```

pi5/etc/systemd/system/star-slam-mvp.service

```

1 [Unit]
2 Description=STAR SLAM MVP stack (supervisor + simple_bridge + LiDAR + SLAM + foxglove)
3 Documentation=file:///workspaces/STAR/docs/ARCHITECTURE.md
4 After=network-online.target tailscaled.service dev-star\x2dmcu.device
5 Wants=network-online.target tailscaled.service
6

```

```

7 [Service]
8 Type=forking
9 User=star
10 Group=star
11 Environment=HOME=/home/star
12 Environment=USER=star
13 Environment=PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
14 # Give USB devices time to enumerate after boot
15 ExecStartPre=/bin/sleep 8
16 ExecStart=/bin/bash /workspaces/STAR/scripts/slam-mvp.sh start
17 ExecStop=/bin/bash /workspaces/STAR/scripts/slam-mvp.sh stop
18 # The script backgrounds each node and returns; forking type matches
19 RemainAfterExit=yes
20 Restart=no
21
22 [Install]
23 WantedBy=multi-user.target

```

pi5/opt/star_cockpit_api/server.py

```

1  #!/usr/bin/env python3
2  """HTTP API backing the Grafana cockpit controls.
3
4  GET /api/state          -> {"autonomy": bool/null, "estop": bool/null, "speed": float/null}
5  POST /api/autonomy/toggle -> flips /star/autonomy_enable, returns new state
6  POST /api/estop/toggle  -> flips /star/estop, returns new state
7  POST /api/speed body: {"value": 0.75} -> publishes /star/speed_scale, returns applied
8
9  Binds to 127.0.0.1:9102; Caddy fronts it as https://star.local/api/*
10 State is learned from the supervisor's latched /star/state/* topics.
11 """
12 import json
13 import subprocess
14 import re
15 import threading
16 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
17 import rclpy
18 from rclpy.node import Node
19 from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy, HistoryPolicy
20 from std_msgs.msg import Bool, Float32
21 from geometry_msgs.msg import Twist
22
23 PORT = 9102
24 LATCHED = QoSProfile(
25     history=HistoryPolicy.KEEP_LAST, depth=1,
26     reliability=ReliabilityPolicy.RELIABLE,
27     durability=DurabilityPolicy.TRANSIENT_LOCAL,
28 )
29
30
31 class CockpitNode(Node):
32     def __init__(self):
33         super().__init__("star_cockpit_api")
34         self._lock = threading.Lock()
35         self._autonomy = None
36         self._estop = None
37         self._last_speed = None
38         self._slam_state_cached = None
39         self._autonomy_pub = self.create_publisher(Bool, "/star/autonomy_enable", 10)
40         self._estop_pub = self.create_publisher(Bool, "/star/estop", 10)
41         self._speed_pub = self.create_publisher(Float32, "/star/speed_scale", 10)
42         self._cmd_vel_pub = self.create_publisher(Twist, "/cmd_vel", 10)
43         self.create_subscription(Bool, "/star/state/autonomy_enable", self._on_auto, LATCHED)
44         self.create_subscription(Bool, "/star/state/estop", self._on_estop, LATCHED)
45         self.get_logger().info(f"cockpit HTTP on 127.0.0.1:{PORT}")
46
47     def _on_auto(self, m):
48         with self._lock: self._autonomy = bool(m.data)
49
50     def _on_estop(self, m):
51         with self._lock: self._estop = bool(m.data)
52
53     def get_state(self):
54         with self._lock:
55             return {"autonomy": self._autonomy, "estop": self._estop, "speed": self._last_speed, "slam":
56                 ↪ self.slam_state()}
57
58     def toggle_autonomy(self):

```

```

58         with self._lock:
59             new = not bool(self._autonomy)
60             self._autonomy_pub.publish(Bool(data=new))
61             return new
62
63     def toggle_estop(self):
64         with self._lock:
65             new = not bool(self._estop)
66             self._estop_pub.publish(Bool(data=new))
67             return new
68
69     def slam_lifecycle(self, transition_id):
70         try:
71             r = subprocess.run(
72                 ["ros2", "service", "call", "/slam_toolbox/change_state",
73                  "lifecycle_msgs/srv/ChangeState",
74                  "{transition: {id: %d}}" % transition_id],
75                 capture_output=True, text=True, timeout=15)
76             ok = "success=True" in r.stdout
77             self._refresh_slam_state()
78             return ok
79         except Exception:
80             return False
81
82     def slam_state(self):
83         return self._slam_state_cached
84
85     def _refresh_slam_state(self):
86         try:
87             r = subprocess.run(
88                 ["ros2", "service", "call", "/slam_toolbox/get_state",
89                  "lifecycle_msgs/srv/GetState"],
90                 capture_output=True, text=True, timeout=5)
91             m = re.search(r"label=\'[a-z_]+\'", r.stdout)
92             new = m.group(1) if m else None
93             with self._lock:
94                 self._slam_state_cached = new
95         except Exception:
96             pass
97
98     def set_cmd_vel(self, linear, angular):
99         m = Twist(); m.linear.x = float(linear); m.angular.z = float(angular)
100         self._cmd_vel_pub.publish(m)
101         return {"linear": m.linear.x, "angular": m.angular.z}
102
103     def set_speed(self, v):
104         v = max(0.0, min(1.0, float(v)))
105         with self._lock:
106             self._last_speed = v
107         self._speed_pub.publish(Float32(data=v))
108         return v
109
110
111     NODE = None # set in main
112
113
114     class H(BaseHTTPRequestHandler):
115         def log_message(self, *a, **k): pass
116
117         def _json(self, code, obj):
118             body = json.dumps(obj).encode()
119             self.send_response(code)
120             self.send_header("Content-Type", "application/json")
121             self.send_header("Content-Length", str(len(body)))
122             self.send_header("Cache-Control", "no-store")
123             self.send_header("Access-Control-Allow-Origin", "*")
124             self.send_header("Access-Control-Allow-Methods", "GET, POST, OPTIONS")
125             self.send_header("Access-Control-Allow-Headers", "Content-Type")
126             self.end_headers()
127             self.wfile.write(body)
128
129         def do_OPTIONS(self):
130             self._json(204, {})
131
132         def do_GET(self):
133             if self.path == "/api/state":
134                 return self._json(200, NODE.get_state())
135             self._json(404, {"error": "not found"})
136

```

```

137 def do_POST(self):
138     if self.path == "/api/autonomy/toggle":
139         return self._json(200, {"autonomy": NODE.toggle_autonomy()})
140     if self.path == "/api/estop/toggle":
141         return self._json(200, {"estop": NODE.toggle_estop()})
142     if self.path == "/api/slam/start":
143         ok = NODE.slam_lifecycle(3)
144         if not ok:
145             NODE.slam_lifecycle(1)
146             ok = NODE.slam_lifecycle(3)
147         return self._json(200 if ok else 500, {"slam": NODE.slam_state(), "ok": ok})
148     if self.path == "/api/slam/stop":
149         ok = NODE.slam_lifecycle(4)
150         return self._json(200 if ok else 500, {"slam": NODE.slam_state(), "ok": ok})
151     if self.path == "/api/cmd_vel":
152         n = int(self.headers.get("Content-Length", 0) or 0)
153         try:
154             body = json.loads(self.rfile.read(n).decode() or "{}")
155             lin = float(body.get("linear", 0))
156             ang = float(body.get("angular", 0))
157         except Exception as e:
158             return self._json(400, {"error": f"bad body: {e}"})
159         return self._json(200, NODE.set_cmd_vel(lin, ang))
160     if self.path == "/api/speed":
161         n = int(self.headers.get("Content-Length", 0) or 0)
162         try:
163             v = float(json.loads(self.rfile.read(n).decode() or "{}").get("value"))
164         except Exception as e:
165             return self._json(400, {"error": f"bad body: {e}"})
166         return self._json(200, {"speed": NODE.set_speed(v)})
167     self._json(404, {"error": "not found"})
168
169
170 def _slam_refresher_loop():
171     import time
172     while True:
173         try:
174             if NODE is not None:
175                 NODE._refresh_slam_state()
176         except Exception:
177             pass
178         time.sleep(4)
179
180 def main():
181     global NODE
182     rclpy.init()
183     NODE = CockpitNode()
184     srv = ThreadingHTTPServer(("100.64.0.6", PORT), H)
185     threading.Thread(target=srv.serve_forever, name="cockpit-http", daemon=True).start()
186     threading.Thread(target=_slam_refresher_loop, name="slam-refresh", daemon=True).start()
187     try:
188         rclpy.spin(NODE)
189     finally:
190         NODE.destroy_node()
191         rclpy.shutdown()
192
193
194 if __name__ == "__main__":
195     main()

```